

Terminalx

```
codio@cleverlogo-eventcricket:~/workspace$ poetry --version
Poetry version 1.1.13
codio@cleverlogo-eventcricket:~/workspace$
```

Guidex

Collapse

Poetry Package Manager

Why Use Poetry?

In the previous module, we saw how powerful `pip` can be; you can perform a wide number of tasks with relative ease. Poetry allows you to perform the same tasks, but it provides a level of automation that streamline working with packages.

Let's start by installing Poetry. The `curl` tool is used to send and receive data. In this instance, we are going to download the `install-poetry.py` script from the given website. We then take this script and pipe (pass) it to `python3`. The end result is that you download and run the `install-poetry.py` script with a single command.

```
curl -sSL https://install.python-poetry.org | python3 -
```

▼ Did you notice?

We did not start a virtual environment before installing Poetry. That is because we want it to be installed at the system level.

In order to verify that the installation worked, check the version of `poetry`.

```
poetry --version
```

You should see output similar to this. **Note**, your version may be newer than the version installed when this tutorial was written.

```
Poetry version 1.1.15
```

Creating a Package

When using `pip`, you are responsible for creating the different directories and files needed for a package. Poetry handles almost all of this for us.

```
poetry new learning-poetry
```

Poetry has created for us a directory for the newly created package. In addition, Poetry also created subdirectories as well as required files. To see what happened during the `new` process, change to the `learning-poetry` directory and call the `tree` command. `tree` prints a hierarchical representation of the directory, where you can see all subdirectories and files.

```
cd learning-poetry && tree
```

You should see the following diagram in the terminal.

```
.
├── learning_poetry
│   ├── __init__.py
├── pyproject.toml
├── README.rst
├── tests
│   ├── __init__.py
│   └── test_learning_poetry.py
```

Poetry creates the `learning_poetry` subdirectory that contains the initialization file. The `pyproject.toml` file contains metadata about the packate — we will talk more about this file later as it is really important. The `README` file is used for documenting the package. Finally, Poetry creates a testing directory. It is setup such that you can run your tests with a simple command.

If you wanted to do the same with `pip`, you would be manually creating these directories and files. You may choose to ignore things like documentation and testing as they are not required. Using Poetry helps to enforce good programming habits.

► Did you notice?

**Try this variation:**

- Create a package with a separate directory for the source code.

```
poetry new --src example2
```

Then print the tree structure of `example2` so that you can compare it to `learning-poetry`.

```
cd example2 && tree
```

▼ What is happening?

Poetry added the `src` directory to the package. This is where all the source code (initialization file and modules) for the package resides. Some programmers prefer to have a `src` directory to keep source files separate from the other files in the package.

**Reading Question**

How do Poetry and `pip` differ when creating a new package?

☐ There is no difference between Poetry and `pip` when creating a new package.

☒ Poetry automates many tasks that have to be done manually in `pip`.

☐ `pip` automates many tasks that have to be done manually in Poetry.

☐ `pip` is faster at creating a new package than Poetry.

The `new` command in Poetry automatically creates the needed files and directories for a package. This would have to be done manually with `pip`.

Check It!